Report of Assignment 2, Foundation of Machine Learning (DA5400)

Harshil Pandey, DA24C006

October 2024

Contents

1 Details of the Submission

- All files having a `.csv` extension are datasets that have been gathered through Kaggle, the exact URLs and usage of data are discussed in one of the subsections below.
- All files with a `.pkl` are learned models that can be directly used to generate predictions without training. The details of generating predictions are discussed in one of the subsections below.
- Libraries Used:
  - `numpy`: for matrix and numerical operations
  - `pandas`: for loading and storing data in matrix format.
  - `matplotlib`: for plotting the graphs
  - `seaborn`: for plotting the graphs
  - `scipy`: for storing the data in a sparse format that saves memory.

  - `joblib`: for converting the learned models into files that may be loaded later to generate predictions.
  - `scikit-learn`: for extracting features from text and `SVM` algorithm.

- `models_helpers.py` file contains the source code which includes procedures that perform loading the data, cross-validation, and classes for two algorithms: Logistic Regression and Naive Bayes.

- The submission includes a comparison of three algorithms, Logistic Regression, Naive Bayes, and SVM.

- `training_demo.py` file contains code that trains the models and presents various visualizations and comparisons that will be discussed in a later section of the report. It is not necessary to run this file to generate predictions. However, the datasets must be present in the working directory before running this file.

- `generate_predictions.py` file contains the code that uses the learned models to generate the predictions based on the test data. This file can be run directly and does not require that the `training_demo.py` file be run before it. It also does not use any datasets as no training is done. After running this file, a file called `predictions.csv` will be generated that will contain the required predictions for the test data.

1.1 Sources and Usage of Data

All datasets are retrieved from Kaggle. The datasets can be extracted from here: Datasets.zip

- `a.csv`: First Dataset Source

  All samples from this dataset are used in training.

- `b.csv`: Second Dataset Source

  All samples from this dataset are used in training.

- `c.csv`: Third Dataset Source

  This dataset is only used for testing.

- `d.csv`: Fourth Dataset Source

  1000 random samples from this dataset are used for training.

- `e.csv`: Fifth Dataset Source

  1000 random samples from this dataset are used for training.

- `f.csv`: Sixth Dataset Source

  This dataset is only used for testing.

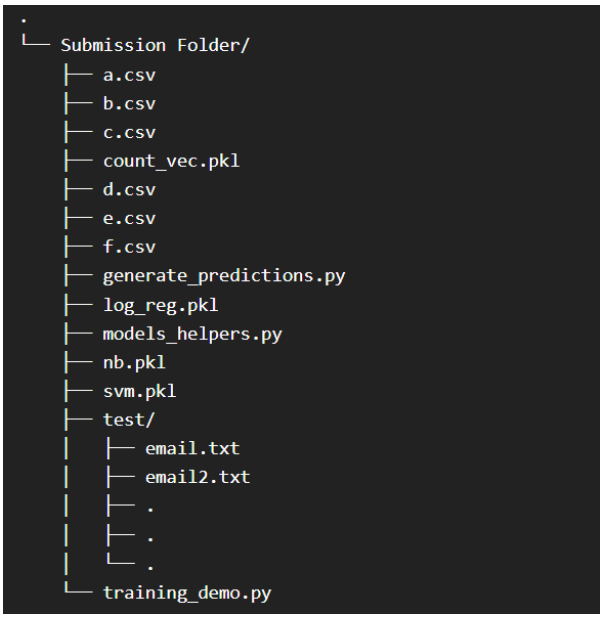

Figure 1: Directory Structure

1.2 Code Setup and Instructions

The directory structure has been explained in *Figure 1*. Here the `test/` directory is the one that contains the emails for which the predictions have to be generated.

Before running any Python file please make sure that all the libraries mentioned above have been installed and the directory structure is satisfied. All Python run commands have to be called from the top-level directory, in this case, `Submission Folder/`. Otherwise reading the files will throw an error.

2 Source Code and Implementation

As discussed above, all the helper functions and algorithm classes are present in `models_helpers.py`.

2.1 Loading Data

The functions used for loading and preprocessing the data are as follows:

- `combine_datasets(combinations, include_points=1000, random_state=None)`

  This function combines datasets gathered from different sources.

  Parameters

  `combinations`: a list that specifies which datasets to combine. Example: [1,3,4] means Datasets 1, 3, and 4 have to be combined.

  `include_points`: specifies how many data points from each dataset and from each class{0, 1} have to be included to form the final dataset. For

example, a value of 500 means that 500 data points will be sampled from each class and from each dataset specified in `combinations`. If a dataset does not have enough data points as specified in any class, then all the data points are returned for that class.

`random_state`: integer for reproducible results

Returns

`pandas.DataFrame` object with the sampled data points.

2.2 Model Evaluation

The functions used for model evaluation processes like cross-validation and reporting the performance metrics of the model are as follows:

- `genrate_splits(n, test_size, random_seed=None)`

  This function yields split indices for training and validation sets of data based on the size of the dataset, and the proportion of points that go into the validation set. Can be used for cross-validation.

  Parameters

  `n`: the size of the dataset (total points)

  `test_size`: the proportion of points that go into the validation subset.

  `random_state`: integer for reproducible results

  Returns

  an iterator that yields a tuple of `numpy` 1-D arrays `(train, test)` where `train` is the integer indices of the training subset, and `test` is the integer indices of the validation subset.

- `cross_validate(X, y, model, test_size, random_seed=None)`

  This function runs the cross-validation algorithm for a given model and dataset.

  Parameters

  `X`: a sparse matrix (`scipy.sparse.csr_matrix`) with data points as rows and features as columns

  `y`: a `numpy 1-D` array with labels 1 or 0.

  `model`: a classification model that supports the `fit` and `predict` method.

  `test_size`: the proportion of data points in a single fold of K-Fold cross-validation.

  `random_state`: integer for reproducible results

  Returns

- `evaluate_model(model, vectorizer)`

  This function prints the performance of the passed model on all datasets. All samples from all datasets are used for performance evaluation

  Parameters

  `model`: a fitted classification model that supports the `predict` method.

`vectorizer`: an `sklearn.feature_extraction.text.CountVectorizer` instance that can extract text features from the raw emails

Returns

a `pandas.Series` object with the dataset number as index and accuracy as the value

2.3 Naive Bayes

Some notations that are used in this subsection:

- x_i is the i^{th} sample of the dataset and $x_i \in \{0,1\}^d$, where d is the number of features(words) in the dataset. The value of the j^{th} feature will be represented by f_j. The label of the i^{th} sample will be represented by y_i.
- $\hat{p} = P(y = 1)$, which is the probability of the label being 1. This is sometimes called the prior probability.
- $\hat{p}_j^1 = P(f_j = 1|y = 1)$, which is the probability of the j^{th} feature being 1 given that the label(y) is 1.
- $\hat{p}_j^0 = P(f_j = 1|y = 0)$, which is the probability of the j^{th} feature being 1 given that the label(y) is 0.
- $P(x_i|y = 1) = \prod_{j=1}^{d} (\hat{p}_j^1)^{f_j}(1 - \hat{p}_j^1)^{(1-f_j)}$, which is the probability to see the i^{th} sample given that the label is 1.
- $P(x_i|y = 0) = \prod_{j=1}^{d} (\hat{p}_j^0)^{f_j}(1 - \hat{p}_j^0)^{(1-f_j)}$, which is the probability to see the i^{th} sample given that the label is 0.

The Naive Bayes algorithm is implemented using the `NaiveBayes` class. Since Naive Bayes acts on the $\{0,1\}^{n\times d}$ input space, using sparse matrix representations saves a lot of memory. Due to this reason, the `NaiveBayes` class also uses sparse matrix representation for performing matrix operations. The instance variables of the class are as follows:

- `p`: This variable represents the value $\hat{p}$ as discussed above.
- `p1`: This is an array of numbers such that the j^{th} value of the array is $\hat{p}_j^1$ as discussed above.
- `p0`: This is an array of numbers such that the j^{th} value of the array is $\hat{p}_j^0$ as discussed above.

The methods of the class are as follows:

- `estimate_params(self, X, y)`: This function estimates the parameters for both classes. The parameters include $\hat{p}_j^1$ and $\hat{p}_j^0$ $\forall j \in \{1, 2 \dots d\}$.

  Parameters

  `X`: a sparse matrix (`scipy.sparse.csr_matrix`) with data points as rows and features as columns

`y`: a `numpy 1-D` array with labels 1 or 0.

Returns

a tuple of `numpy 1-D` arrays `(a0, a1)`, where `a0` is in same format as `p0` and `a1` is in the same format as `p1`.

The estimations are made using the formula: $\hat{p}_j^y = \frac{\sum_{i=1}^{n} f_j \mathbf{i}(y_i = y)}{\sum_{i=1}^{n} \mathbf{i}(y_i = y)}$

Programmatically, this is easy to do, since using y, we first extract the indices of points whose label is say 1, and using these, we index on the rows of `X` to create the necessary subset of samples. Then, the mean of the j^{th} column of the subset matrix gives us $\hat{p}_j^1$.

- `add_smoothing(self, X, y)`: This function adds smoothing data points to the dataset passed.

  Parameters

  `X`: a sparse matrix (`scipy.sparse.csr_matrix`) with data points as rows and features as columns

  `y`: a `numpy 1-D` array with labels 1 or 0.

  Returns

  a tuple `(new_data_matrix, new_labels)`, where `new_data_matrix` is a `scipy.sparse.csr_matrix` object with smoothing rows added and `new_labels` is a `numpy 1-D` array with labels 1 and 0 having the smoothing labels added.

  Suppose, the j^{th} feature is 0 in all the training samples. Then the values of $\hat{p}_j^1$ and $\hat{p}_j^0$ will be 0. Now suppose, a test sample has the j^{th} feature's value as 1. In this case, both $P(x_{test}|y = 1)$ and $P(x_{test}|y = 0)$ will be 0. So, we will not be able to predict the label for such a test sample. To avoid this we add smoothing, in which we add four extra data points to the dataset, two of which have all feature values as 1 with 0 and 1 as the corresponding labels. The other two data points have all feature values as 0 with 0 and 1 as the corresponding labels.

  This is an important step in the algorithm as it makes sure that the model will be able to generate a prediction for all possible inputs.

- `fit(self, X, y)`: This function fits the model by adding smoothing and estimating the required parameters.

  Parameters

  `X`: a sparse matrix (`scipy.sparse.csr_matrix`) with data points as rows and features as columns

  `y`: a `numpy 1-D` array with labels 1 or 0.

  Using the `add_smoothing(self, X, y)` and `estimate_params(self, X, y)` functions, this function calculates the required parameters and stores them in the corresponding instance variables.

- `predict(self, X)`: This functions predicts the labels for the data points in `X`.

  Parameters

  `X`: a sparse matrix (`scipy.sparse.csr_matrix`) with data points as rows and features as columns

  Returns

  a `numpy 1-D` array with corresponding predicted labels 0 or 1.

  The prediction formula for a given sample point is as follows:

  Predict 1 if:

$$\sum_{j=1}^{d} f_j \log\left(\frac{\hat{p}_j^1(1-\hat{p}_j^0)}{\hat{p}_j^0(1-\hat{p}_j^1)}\right) + \sum_{j=1}^{d} \log\left(\frac{1-\hat{p}_j^1}{1-\hat{p}_j^0}\right) + \log\left(\frac{\hat{p}}{1-\hat{p}}\right) \geq 0$$

  This can be converted into a form:

$$w^T x + b \geq 0,$$
$$\text{where } w_j = \log\left(\frac{\hat{p}_j^1(1-\hat{p}_j^0)}{\hat{p}_j^0(1-\hat{p}_j^1)}\right)$$
$$\text{and } b = \sum_{j=1}^{d} \log\left(\frac{1-\hat{p}_j^1}{1-\hat{p}_j^0}\right) + \log\left(\frac{\hat{p}}{1-\hat{p}}\right)$$

  Programmatically, this becomes easy, as a simple matrix multiplication generates the predictions of the test dataset. Precisely speaking, if

$$z = Xw + \hat{b}$$

  where X is the test data matrix, such that each row is a sample, and $\hat{b}$ is an n dimensional vector with each element as b then the prediction is:

  Predict 1 for i^{th} sample if $z_i \geq 0$

2.4 Logistic Regression

Some notations that are used in this subsection:

- x_i is the i^{th} sample of the dataset and $x_i \in \mathbf{R}^d$, where d is the number of features(words) in the dataset. The j^{th} component of the sample will represent the number of times the corresponding word has occurred in the mail. The label of the i^{th} sample will be represented by y_i.
- $\sigma(z) = \frac{1}{1+e^{-z}}$, where $\sigma(z)$ is the sigmoid function.
- θ is the d dimensional weight vector that the Logistic Regression algorithm learns. The j^{th} component(θ_j) is the weight corresponding to the j^{th} feature.
- $\hat{y}_i$ is the prediction of the i^{th} sample. The formula for calculating $\hat{y}_i$ is:

$$\hat{y}_i = \sigma(x_i \cdot \theta)$$
$$\hat{y}_i = P(y_i = 1 | x_i = 1)$$

- $L(\theta)$ is the loss function with respect to a weight vector θ. The objective of the algorithm is to minimize this loss function with respect to the training dataset by learning the suitable weight vector.
- $\nabla_\theta L(\theta)$ is the gradient of the loss function with respect to a weight vector θ.

Unlike Naive Bayes, the input space of Logistic Regression is $\mathbf{R}^{n \times d}$, but due to the nature of the problem, most of the features for each sample will be 0 as not every word will be present in each mail. Hence, in this case, the sparse representation will also save a lot of memory. The algorithm is implemented using the `LogReg` class which uses sparse matrices for all the matrix calculations. To obtain the solution to the problem, we use Gradient Descent on the loss function($L(\theta)$). The instance variables of the class are:

- `iterations`: the number of iterations to perform to obtain the gradient descent solution of the problem.
- `step_size`: The learning rate of gradient descent.
- `coef`: This is an array such that the j^{th} element of the array is θ_j as discussed above.

The methods of the class are as follows:

- `sigmoid(self, z)`: This function calculates the sigmoid($\sigma(z)$) of each element of `z`.

  Parameters

  `z`: a `numpy 1-D` array having float values

  Returns

  a `numpy 1-D` array having elements as sigmoid of corresponding elements of `z`.

- `loss(self, X, y)`: This function returns the cross-entropy loss($L(\theta)$) of the dataset with the learned weights.

  Parameters

  `X`: a sparse matrix (`scipy.sparse.csr_matrix`) with data points as rows and features as columns

  `y`: a `numpy 1-D` array with labels 1 or 0.

  Returns

  The cross-entropy loss(float) of the dataset with the learned weights.

  The formula for the loss function is:

$$L(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \left(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right)$$

- `gradient(self, X, y)`: This function calculates the gradient vector($\nabla_\theta L(\theta)$) of the dataset with the learned weights.

  Parameters

`X`: a sparse matrix (`scipy.sparse.csr_matrix`) with data points as rows and features as columns

`y`: a `numpy 1-D` array with labels 1 or 0.

Returns

a `numpy 1-D` array which is the gradient vector.

The formula for calculating the gradient vector is:

$$\nabla_\theta L(\theta) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)x_i$$

Programmatically, this is easy to calculate. Let D be a $n \times n$ diagonal matrix such that $D_{ii} = \hat{y}_i - y_i$. Then the mean of the j^{th} column of the matrix DX will be $\nabla_\theta L(\theta)$.

- `gradient_descent(self, X, y)`: This function runs the gradient descent algorithm on the dataset with initial weights as 0. It updates the `coef` instance variable accordingly.

  Parameters

  `X`: a sparse matrix (`scipy.sparse.csr_matrix`) with data points as rows and features as columns

  `y`: a `numpy 1-D` array with labels 1 or 0.

  The gradient descent update rule is as follows:

  $$\theta^{(t+1)} = \theta^{(t)} - \alpha\nabla_{\theta^{(t)}} L(\theta)$$

  where $\theta^{(t)}$ is the weight vector obtained after t iterations, α is the learning rate and, $\theta^{(0)} = \mathbf{0}$

- `fit(self, X, y)`: This function fits the model on the passed dataset by calling the `gradient_descent` method.

  Parameters

  `X`: a sparse matrix (`scipy.sparse.csr_matrix`) with data points as rows and features as columns

  `y`: a `numpy 1-D` array with labels 1 or 0.

- `predict(self, X)`: This functions predicts the labels for the data points in `X`.

  Parameters

  `X`: a sparse matrix (`scipy.sparse.csr_matrix`) with data points as rows and features as columns

  Returns

  a `numpy 1-D` array with corresponding predicted labels 0 or 1.

  The prediction formula for a given sample point is:

  Predict 1 if: $\hat{y}_i \geq 0.5$

3 Observations and Inferences

All samples from `a.csv` and `b.csv` were used. 1000 samples each were used from `d.csv` and `e.csv` such that the class proportion was 0.5 in both the subsets. 75% of this combined dataset was used for initial training and cross-validation. The remaining was treated as the test dataset. After deciding the best hyper-parameters of the models based on the cross-validation and testing performance, the models were re-trained on the best hyper-parameters on the whole combined dataset. Then the final evaluation was made based on the performance on entire individual datasets from all six sources as discussed in the first section of the report.

For converting the raw emails into features, I have used `CountVectorizer` which is a `scikit-learn` utility that converts the raw emails into a data matrix such that each row is a sample and each column represents a word that may or may not occur in the mail. If the word occurs in the mail then the feature value is the count of occurrence in the mail, otherwise, the value is 0. `CountVectorizer` also supports sparse output which helps us save memory, furthermore, it has a functionality to remove 'stop-words' from the mails. A 'stop-word' is a word that may not be useful in predicting the label of the mail such as 'the', 'a', 'an', 'to', etc. These words are present in almost all emails as they are part of English syntax and hence do not provide any useful information in classification.

3.1 Naive Bayes

The results after running cross-validation on Naive Bayes algorithm are as follows:

Best Prior Value: 0.018
Training Data Accuracy: 97.171%
Testing Data Accuracy: 86.572%

Figure 2 plots the cross-validation results on the Naive Bayes algorithm. 50 different values from 0 to 0.9 were taken as priors for cross-validation. The best prior value of 0.018 indicates that even though the training samples approximately have equal class proportion, the probability of an email being spam is low.

3.2 Logistic Regression

The results after running cross-validation on the Logistic Regression algorithm are as follows:

Best Learning Rate: 0.003
Training Data Accuracy: 98.821%
Testing Data Accuracy: 88.693%

20 different values between 0.0001 to 0.01 were taken for the learning rate parameter of the algorithm. *Figure 3* plots the cross-validation results on Logistic Regression. We can see that the accuracy peeks at 0.003. This can be attributed to the fact that when the learning rate is very small, around 0.0005, then the algorithm can't converge as the updates are small. When the learning

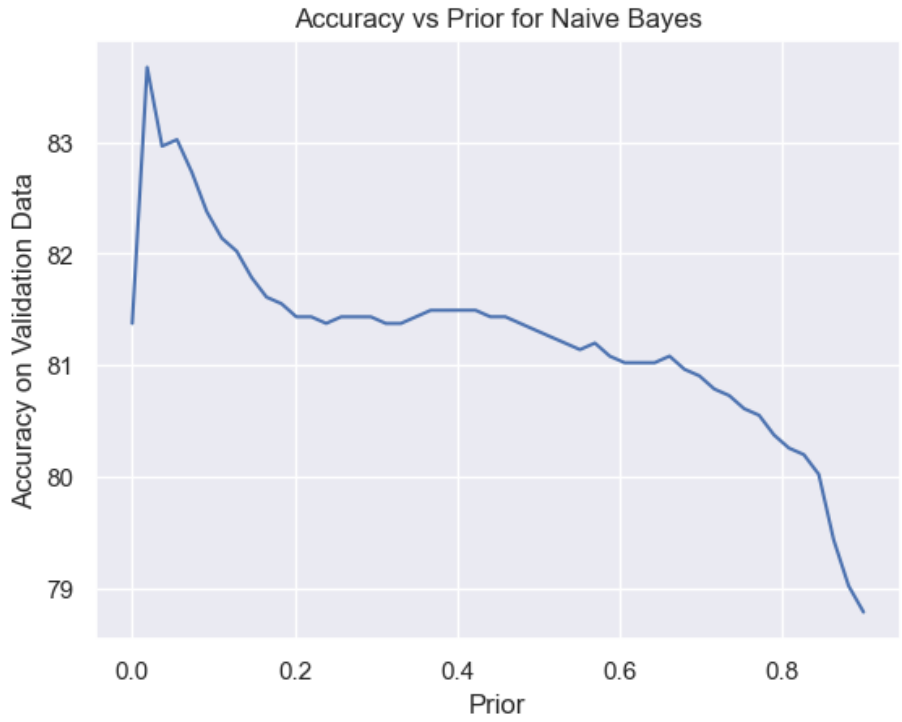

Figure 2: Cross-Validation on Naive Bayes

rate is larger, around 0.006, then the updates are very large and the algorithm is overshooting.

We can also see that the performance of Logistic Regression is slightly better than Naive Bayes.

3.3 SVM

The results after running cross-validation on the SVM algorithm are as follows:

Best Regularization Parameter: 44.474
Training Data Accuracy: 97.996%
Testing Data Accuracy: 86.926%

In `scikit-learn` the smaller the value of the regularization parameter, the stronger the regularization. We can clearly see in *Figure 4* that smaller values of the parameter lead to poor accuracies as it may lead to under-fitting due to very strong regularization. The higher values of the parameter yield similar results.

We can also see that SVM performs better than Naive Bayes but worse than Logistic Regression.

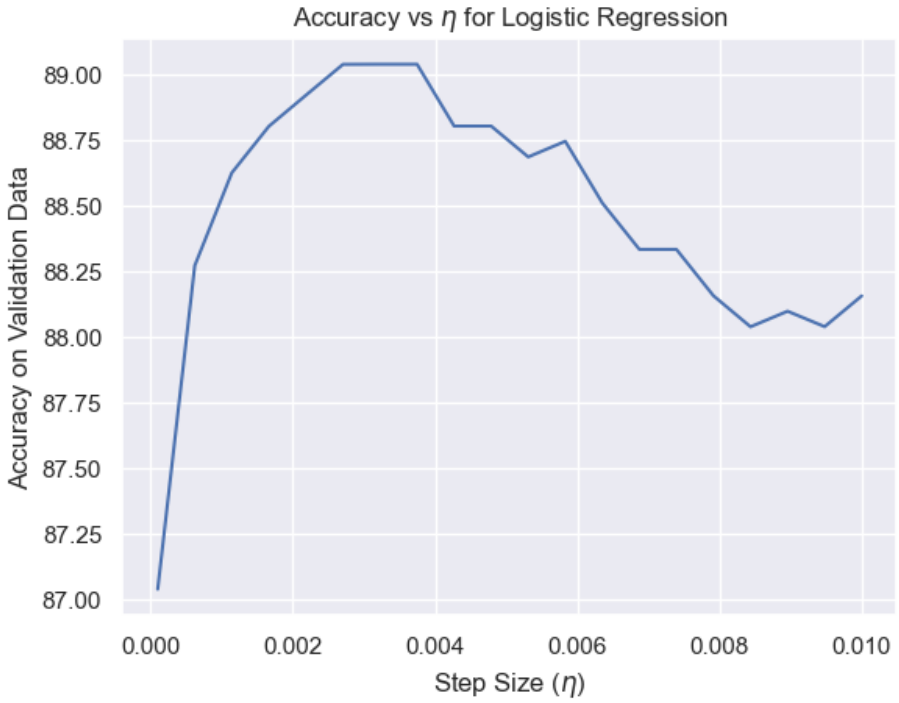

Figure 3: Cross-Validation on Logistic Regression

3.4 Performance on all Datasets

The table can be used to refer to the information related to the dataset along with the model performance. From these plots we can infer the following points:

- Naive Bayes performs very well on all the datasets, even better than Logistic Regression and SVM, except in the case of Dataset 4, where it performs poorly. The dip in performance is visible in *Figure 6*. This is unusual since Dataset 4 was actually used for training. It may be because the sample we took may not be completely representative of the whole dataset. However, Logistic Regression performs well on the same dataset.
- SVM also seems to perform very well on all datasets except Dataset 3 and Dataset 6. Both these datasets were not included in the training process. We can attribute this behavior to the fact that it over-fit the training data. This dip in performance is also observed in the graph.
- Logistic Regression seems to be the only model that performs well on all the datasets. It is the only model that scores an accuracy of over 80% on all the models. The line trend also vividly explains this since there is no significant dip in its performance on any dataset.

For this reason, I have decided to use Logistic Regression as my final model of choice. The `genrate_predictions.py` file will use the Logistic Regression model to make predictions.

Figure 4: Cross-Validation on SVM

```
Naive Bayes
        Accuracy on Dataset 1 : 83.240
        Spam % : 44.134
        Size : 179
-------
        Accuracy on Dataset 2 : 97.619
        Spam % : 30.952
        Size : 84
-------
        Accuracy on Dataset 3 : 86.031
        Spam % : 28.252
        Size : 5412
-------
        Accuracy on Dataset 4 : 68.665
        Spam % : 13.406
        Size : 5572
-------
        Accuracy on Dataset 5 : 89.996
        Spam % : 52.620
        Size : 83448
-------
        Accuracy on Dataset 6 : 86.900
        Spam % : 47.300
        Size : 193850
-------
Logistic Regression
        Accuracy on Dataset 1 : 92.737
        Spam % : 44.134
        Size : 179
-------
        Accuracy on Dataset 2 : 96.429
        Spam % : 30.952
        Size : 84
-------
        Accuracy on Dataset 3 : 83.721
        Spam % : 28.252
        Size : 5412
-------
        Accuracy on Dataset 4 : 86.683
        Spam % : 13.406
        Size : 5572
-------
        Accuracy on Dataset 5 : 90.581
        Spam % : 52.620
        Size : 83448
-------
        Accuracy on Dataset 6 : 86.013
        Spam % : 47.300
        Size : 193850
-------
SVM
        Accuracy on Dataset 1 : 99.441
        Spam % : 44.134
        Size : 179
-------
        Accuracy on Dataset 2 : 94.048
        Spam % : 30.952
        Size : 84
-------
        Accuracy on Dataset 3 : 61.308
        Spam % : 28.252
        Size : 5412
-------
        Accuracy on Dataset 4 : 98.475
        Spam % : 13.406
        Size : 5572
-------
        Accuracy on Dataset 5 : 86.102
        Spam % : 52.620
        Size : 83448
-------
        Accuracy on Dataset 6 : 78.651
        Spam % : 47.300
        Size : 193850
-------
```

Figure 5: Model Comparison Table

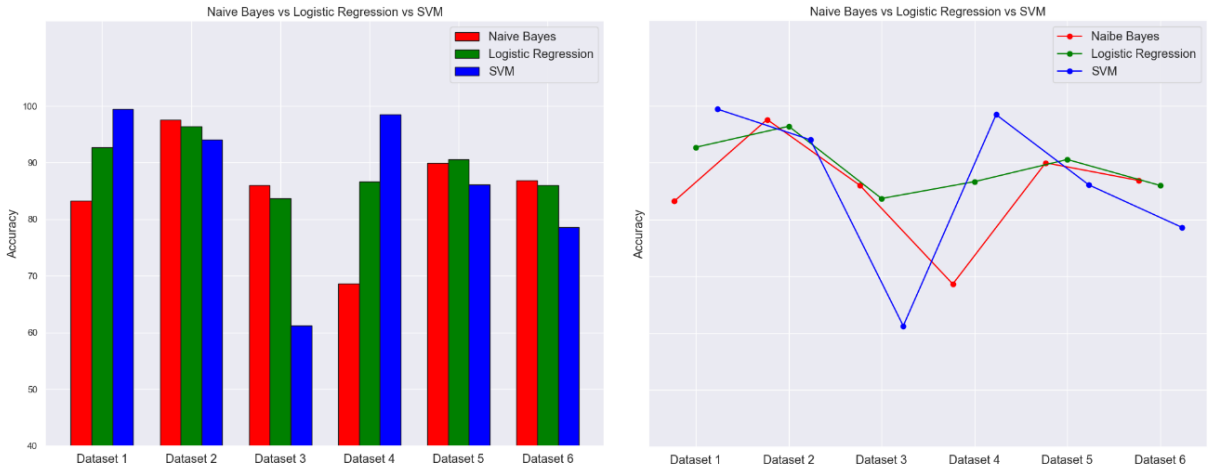

Figure 6: Model Comparison Plots